

**SIMATS
ENGINEERING
ASSESSMENT TOOL 2**

**COURSE NAME: OBJECT ORIENTED ANALYSIS AND
DESIGN COURSE CODE: CSA1104**

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 40%

QUESTIONS:
5

**Total
Marks:40**

Annexure A

Industry Problem Solving Task

Code of Conduct:

I, B.Harshitha (192420006) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Harshitha

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the

presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. Smart Banking and Financial Management System

The Smart Banking and Financial Management System manages customer accounts, fund transfers, loans, bill payments, transaction history, and fraud notifications using a layered object-oriented architecture connected to an OODBMS. The proposed system specifically uses Factory, Strategy, Observer, and Repository patterns.

Object-Oriented Principles Encapsulation:

Banking information such as account balance, customer details, loan information, and transaction records is stored inside objects and accessed through controlled methods. This improves security by preventing unauthorized modification.

Abstraction:

Complex operations such as fund transfer, loan calculation, and database access are represented through simple interfaces.

Inheritance:

A common Account class can be extended into SavingsAccount, CurrentAccount, and other account types. This promotes code reuse.

Polymorphism:

Different account types and payment strategies can implement the same operations differently without changing the rest of the system.

Layered Architecture

1. **Presentation Layer:** Provides interfaces for login, account details, payments and transactions.
2. **Service Layer:** Handles fund transfers, loan processing, bill payments and validation.
3. **Domain Layer:** Contains objects such as Customer, Account, Loan and Transaction.
4. **Data Access Layer:** Uses repositories to store and retrieve objects from the OODBMS.

The communication flow is:

**Presentation Layer → Service Layer → Domain Layer → Repository/Data Access Layer
→ OODBMS**

Design Patterns

Factory Pattern: Creates different account objects without tightly coupling the application to specific account classes.

Strategy Pattern: Supports different loan-interest and payment calculation policies. New policies can be introduced without modifying existing processing logic.

OODBMS Interaction

The Repository layer converts domain operations into OODBMS persistence operations. Objects such as Customer, Account, Loan, and Transaction can be stored directly while preserving their relationships.

For example:

Customer Request → Banking Service → Account Object → Account Repository → OODBMS

During retrieval:

OODBMS → Repository → Persistent Object → Service Layer → User Interface

Analysis

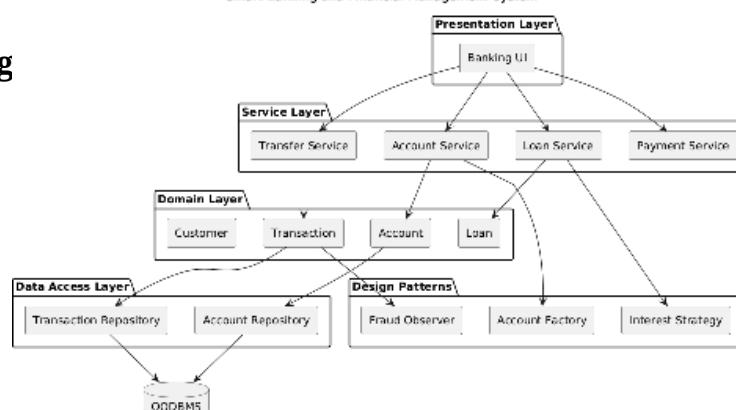
This architecture provides:

- **Scalability:** New services such as digital wallets and investment management can be added as separate modules.
- **Security:** Encapsulation and controlled service/repository access protect banking information.
- **Flexibility:** Strategy and Factory patterns allow algorithms and account types to change independently.
- **Maintainability:** Separation of layers reduces dependencies and makes testing and modification easier.

Conclusion

The combination of object-oriented principles, layered architecture, design patterns, and OODBMS persistence produces a secure, scalable and maintainable banking solution that can accommodate future financial services.

UML Diag



2. Smart Retail and Inventory Management System

The Smart Retail and Inventory Management System handles products, suppliers, customers, purchase orders, sales transactions and warehouse stock. It uses MVC, Factory, Observer and DAO patterns with an OODBMS for persistent inventory management.

Object-Oriented Principles

Encapsulation: Product price, quantity, supplier information and stock levels are maintained inside their respective objects.

Abstraction: Users interact with simple operations such as `addProduct()`, `updateStock()` and `placeOrder()` without knowing internal implementation.

Inheritance: A general Product class can be extended into categories such as Electronics, Groceries and Clothing.

Polymorphism: Different product categories can calculate discounts, taxes, or stock requirements differently through common interfaces.

Layered Architecture

Presentation Layer: Displays product, sales and inventory information.

Business Layer: Performs sales processing, stock validation, purchase-order management and inventory calculations.

Domain Layer: Contains Product, Supplier, Customer, Order and Inventory objects.

Data Access Layer: Uses DAO classes to communicate with the OODBMS.

Communication follows:

Presentation → Business → Domain → DAO → OODBMS

Design Patterns

MVC Pattern: Separates the retail user interface into Model, View and Controller, allowing the interface and business logic to be modified independently.

Factory Pattern: Creates different product-category objects through a common product-creation mechanism.

Observer Pattern: Automatically alerts managers when inventory reaches its minimum stock threshold.

DAO Pattern: Encapsulates OODBMS operations such as saving, retrieving, updating and deleting persistent inventory objects.

OODBMS Interaction

When a sale occurs, the business layer updates the corresponding Product and Inventory objects. The DAO layer stores the updated objects in the OODBMS.

Example:

Sales UI → Sales Controller → Inventory Service → Inventory Object → InventoryDAO → OODBMS

For retrieval:

OODBMS → InventoryDAO → Inventory Object → Business Layer → Dashboard

Analysis

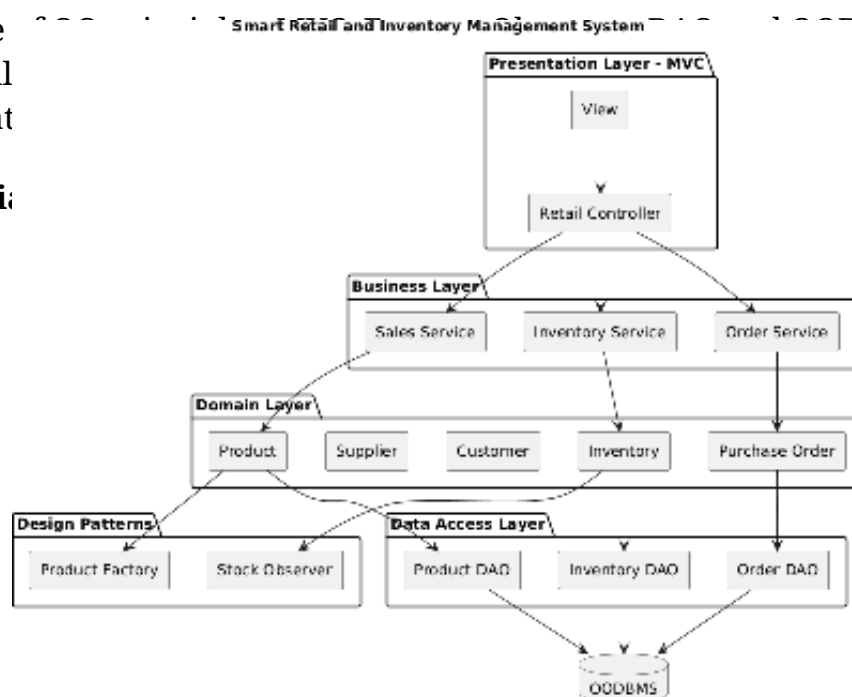
The architecture improves:

- **Flexibility:** New product categories can be added through the Factory Pattern.
- **Scalability:** Stock prediction and robotic warehouse modules can be added without redesigning the complete system.
- **Maintainability:** MVC and layered separation reduce coupling.
- **Efficiency:** DAO provides organized and reusable persistence operations.
- **Reliability:** Observer alerts prevent stock shortages.

Conclusion

The use of a modular architecture results in automated BMS results in

UML Diagram



3. Smart Hotel and Hospitality Management System

The Smart Hotel Management System supports guest registration, reservations, check-in/check-out, payment, room services and customer feedback. It uses Strategy, Factory, Observer and Repository patterns with an OODBMS.

Object-Oriented Principles

Encapsulation: Guest details, reservation information, payment information and room status are protected inside domain objects.

Abstraction: Complex reservation and payment processes are provided through simple service interfaces.

Inheritance: A parent Room class may be extended into StandardRoom, DeluxeRoom and SuiteRoom.

Polymorphism: Different room, service and pricing objects can perform common operations using different implementations.

Layered Architecture

Presentation Layer: Provides reservation, check-in, payment and service interfaces.

Service Layer: Handles booking, room availability, check-in/check-out, payment and customer services.

Domain Layer: Contains Guest, Room, Reservation, Payment and RoomService.

Data Access Layer: Uses repositories for persistent data management. Flow:

Presentation → Service → Domain → Repository → OODBMS

Design Patterns

Strategy Pattern: Implements different pricing and discount policies such as seasonal pricing, membership discounts and promotional pricing.

Factory Pattern: Creates different room and hotel-service objects.

Observer Pattern: Sends automatic notifications for reservation confirmation, room-service updates and related events.

Repository Pattern: Provides a clean interface between service components and persistent hotel objects.

Reservation and OODBMS Processing

During reservation:

Guest UI → Reservation Service → Availability Check → Reservation Object → Reservation Repository → OODBMS

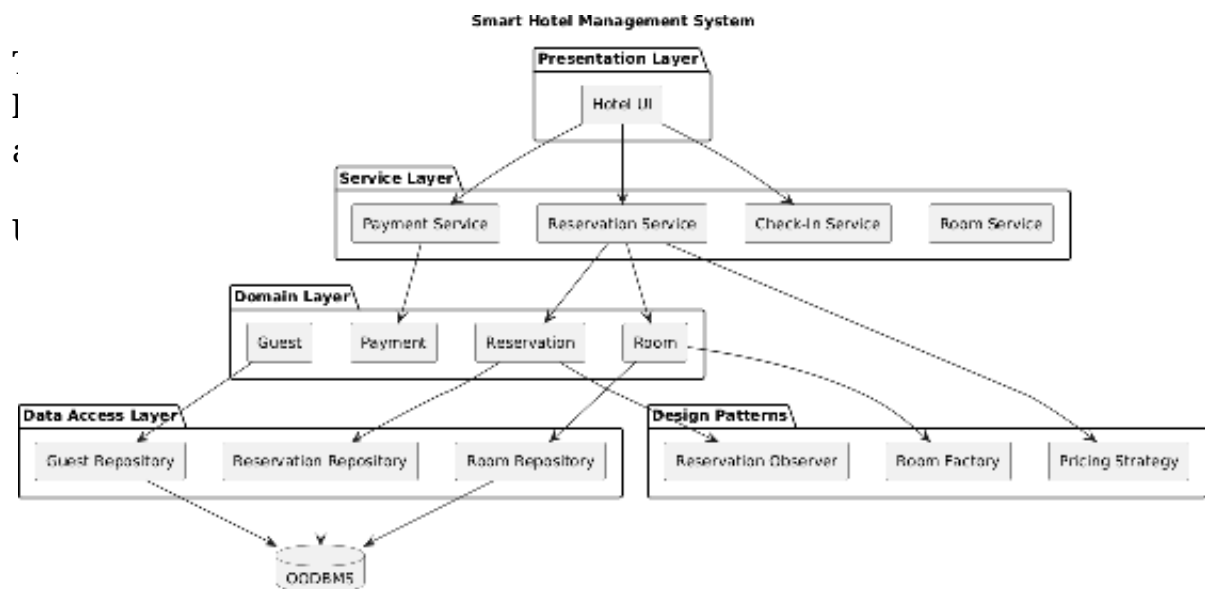
The OODBMS stores persistent objects and their relationships, such as a guest being associated with a reservation and the reservation being associated with a room.

During retrieval, the Repository obtains the required reservation or room object from the OODBMS and returns it to the service layer.

Analysis

- **Flexibility:** Pricing strategies can be replaced without modifying reservation logic.
- **Reusability:** Common Room and Service classes reduce duplicate code.
- **Maintainability:** Layer separation allows each module to be modified independently.
- **Extensibility:** Smart-room automation and personalized guest-service modules can be incorporated later.
- **Reliability:** Observer notifications keep guests and hotel staff updated.

Conclusion



4. Smart Logistics and Delivery Management System

The Smart Logistics and Delivery Management System manages shipments, warehouse operations, vehicles, delivery tracking and customer notifications. It applies Strategy, Factory, Observer and DAO patterns and maintains shipment and vehicle information in an OODBMS.

Object-Oriented Principles

Encapsulation: Shipment details, vehicle status, delivery locations and customer information are maintained as protected object data.

Abstraction: Operations such as shipment creation, route selection and delivery tracking are provided through service interfaces.

Inheritance: A general Shipment class can support specialized shipment classes such as ExpressShipment and StandardShipment.

Polymorphism: Different shipment and route-selection strategies can implement common operations differently.

Layered Architecture

Presentation Layer: Provides shipment booking, tracking and customer-notification interfaces.

Business Layer: Handles shipment processing, route calculation, vehicle allocation and delivery management.

Domain Layer: Contains Shipment, Vehicle, Warehouse, Delivery and Customer.

Data Access Layer: Uses DAO objects for persistence. Flow:

Presentation → Business → Domain → DAO → OODBMS

Design Patterns

Strategy Pattern: Selects an appropriate delivery-route algorithm depending on distance, cost, traffic, or delivery priority.

Factory Pattern: Creates different shipment types without directly coupling clients to concrete classes.

Observer Pattern: Automatically informs customers and managers whenever delivery status changes.

DAO Pattern: Handles persistent shipment, vehicle and delivery objects.

OODBMS Interaction

For shipment registration:

User Interface → **Shipment Service** → **Shipment Object** → **ShipmentDAO** → **OODBMS**

For delivery tracking:

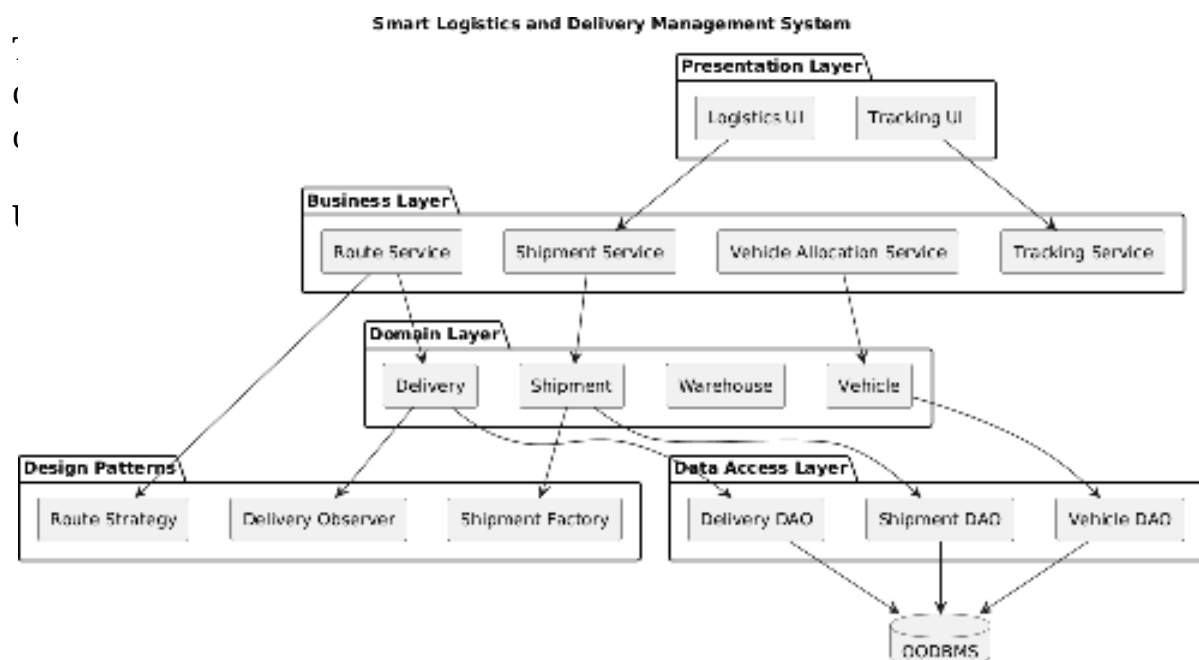
OODBMS → **DeliveryDAO** → **Delivery Object** → **Tracking Service** → **Customer Interface**

The OODBMS preserves relationships between shipment, delivery, vehicle and warehouse objects.

Analysis

- **Adaptability:** Route algorithms can be changed using the Strategy Pattern.
- **Scalability:** Autonomous vehicles and AI route optimization can be introduced as additional modules.
- **Maintainability:** Business and database operations remain separated.
- **Real-time communication:** Observer-based notifications inform customers about delivery changes.
- **Reuse:** Factory-based shipment creation removes repeated object-creation logic.

Conclusion



5. Smart Energy Management System

The Smart Energy Management System monitors electricity consumption, smart meters, energy usage and abnormal-consumption alerts. It contains presentation, business, domain and data-access layers and uses Observer, Strategy, Factory and Repository patterns.

Object-Oriented Principles

Encapsulation: Smart-meter readings, customer consumption and energy parameters are maintained as protected object attributes.

Abstraction: Energy-monitoring and analysis functions are exposed through high-level services while internal calculations remain hidden.

Inheritance: A common SmartMeter class can support different types of meter objects.

Polymorphism: Different energy-optimization strategies can implement a common optimization operation using different algorithms.

Layered Architecture

Presentation Layer: Displays dashboards, consumption reports and alerts.

Business Layer: Performs consumption analysis and energy optimization.

Domain Layer: Contains energy rules and objects such as SmartMeter, EnergyReading, Consumer and Alert.

Data Access Layer: Uses repositories to communicate with the OODBMS. Flow:

Dashboard → Business Layer → Domain Objects → Repository → OODBMS

Design Patterns

Observer Pattern: Observers are automatically notified when abnormal energy consumption is detected.

Strategy Pattern: Different energy-optimization algorithms can be selected dynamically.

Factory Pattern: Creates different smart-meter objects through a common creation interface.

Repository Pattern: Manages storing and retrieving persistent energy objects without exposing database-specific operations to the business layer.

OODBMS Interaction

A smart meter produces consumption readings which are represented as domain objects.

Smart Meter → Energy Service → EnergyReading Object → Energy Repository → OODBMS

For analysis:

OODBMS → Repository → Energy Objects → Analysis Service → Dashboard

Relationships between meters, consumers, readings and alerts can therefore remain represented through persistent objects.

Analysis

The solution improves:

- **Scalability:** More smart meters and customers can be supported through independent objects and services.
- **Extensibility:** Renewable-energy monitoring and AI-based prediction modules can be incorporated later.
- **Maintainability:** Each layer performs a clearly defined responsibility.
- **Flexibility:** Strategy allows optimization algorithms to be replaced easily.
- **Monitoring efficiency:** Observer-based alerts automatically react to abnormal consumption.
- **Data management:** Repository-based persistence isolates OODBMS operations from energy-analysis logic.

Conclusion

The Smart Energy Management System demonstrates how object-oriented

